

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 3

The Command Line: Shell Architecture, Path Expansion, I/O Redirection, and Stream Filtering Pipelines in Linux

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **2**

1 THEORETICAL FOUNDATIONS OF SHELL ARCHITECTURE AND STREAM PIPELINES 2

1.1 Shell Execution Lifecycle and Process Creation 3

1.2 Virtual File Descriptors and Kernel I/O Redirection 3

1.3 Inter-Process Communication via Unidirectional Pipes 4

1.4 Lexical Analysis, Expansions, and Quoting Rules 4

2 EMPIRICAL INVESTIGATION AND PRACTICAL IMPLEMENTATION 5

2.1 Navigation, Command Resolution, and Diagnostic Discovery 6

2.2 Shell Expansions, Quoting Semantics, and Globbing Mechanics 7

2.3 Stream Redirection, Chaining Logic, and Process Job Control 10

2.4 Analytical Stream Processing with One-Liner Pipelines 12

2.5 Automated Administrative Reporting Script 15

CONCLUSIONS **17**

REFERENCES **18**

ANNEX A: SOURCE CODE LISTINGS **19**

INTRODUCTION

The command-line interface constitutes the fundamental operational boundary between software engineers and the UNIX operating system. While graphical user interfaces present pre-packaged interactive abstractions, the terminal shell exposes direct programmatic access to kernel services, process control primitives, and the virtual filesystem hierarchy. In modern POSIX-compliant environments such as GNU/Linux, the command language is neither a passive text dispatcher nor an arbitrary batch parser; rather, it is a Turing-complete programming environment designed around the foundational Unix philosophy of modularity, composability, and clean stream isolation.

At the core of this operational model lies the concept of uniform data streams. By abstracting standard input, standard output, and standard error as universal byte streams associated with integer file descriptors, the operating system decouples data producers from data consumers. Small, specialized utilities, such as search filters, field selectors, line sorters, and tallying counters, can be interconnected via unidirectional kernel IPC channels known as pipes. Consequently, complex diagnostic tasks, forensic log inspections, and batch data processing operations can be articulated concisely in a single command line without requiring custom compiled binaries or intermediate physical storage.

The primary objective of this laboratory work is to dissect the internal mechanics of the Bash shell from the perspective of an operating systems engineer. The practical investigation encompasses five structured milestones: evaluating interactive navigation and command resolution pathways, modeling the parsing and expansion stages through which input lines transition prior to kernel execution, validating file descriptor manipulation and asynchronous job scheduling, formulating atomic stream filtering pipelines to analyze artificial server workload logs, and authoring a robust, automated administrative reporting utility. The complete codebase, script implementations, and verification logs are maintained in the GitHub repository accessible via `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL FOUNDATIONS OF SHELL ARCHITECTURE AND STREAM PIPELINES

1.1 Shell Execution Lifecycle and Process Creation

The shell operates as an iterative command loop driven by the classic read-eval-print sequence. In user space, the shell process reads character input from the controlling terminal terminal line discipline, parses lexical tokens, performs parameter and pathname substitutions, and dispatches the resulting executable command to the operating system kernel.

Unlike integrated language runtimes where subroutines execute within a single address space, general-purpose command execution in Unix relies on the explicit fork-exec process creation model. When a non-builtin executable is invoked, the parent shell issues the `fork()` system call. The kernel duplicates the calling process, creating a child process with an isolated virtual memory address space, identical memory page mappings (initially protected via copy-on-write mechanisms), and duplicated file descriptor tables. The child process then invokes the `execve()` system call, replacing its address space, stack, heap, and CPU register state with the binary image loaded from the filesystem. The parent shell typically invokes `waitpid()` to suspend execution until the child process terminates, retrieving its final numeric exit status.

Crucially, certain commands cannot execute inside a separate child process. The most prominent example is `cd` (change directory). In Unix, the current working directory is an internal attribute of each process maintained in kernel memory within the task structure. If the shell were to spawn an external process to execute `cd`, the child process would invoke the `chdir()` system call, successfully altering its own working directory, and immediately terminate. The parent shell process would remain in its original directory. Therefore, commands that modify the execution context or environment of the shell must be implemented as internal shell built-ins.

1.2 Virtual File Descriptors and Kernel I/O Redirection

Every process in a Linux system possesses an open file table indexed by small non-negative integers termed file descriptors. By POSIX convention, every standard process initializes with three predefined streams: standard input (descriptor 0), standard output (descriptor 1), and standard error (descriptor 2).

Table 1.1 – Standard POSIX File Descriptors

Descriptor	POSIX Name	Default Binding	Operational Purpose
0	`STDIN_FILENO`	Keyboard / Controlling TTY	Input byte stream ingested by process
1	`STDOUT_FILENO`	Display Screen / Terminal	Primary functional output stream
2	`STDERR_FILENO`	Display Screen / Terminal	Unbuffered error and diagnostic output

Stream redirection is achieved by manipulating this file descriptor table prior to invoking `execve()`. When the shell encounters an output redirection token such as `> filename`, it opens or creates the target file using the `open()` system call with flags `O_WRONLY | O_CREAT | O_TRUNC`. Subsequently, it executes the `dup2(target_fd, 1)` system call. This kernel operation closes file descriptor 1 and duplicates the open file description of the target file into slot 1. When the child executable later writes data to standard output via the `write(1, buffer, count)` system call, the bytes flow transparently into the designated disk file without the application requiring any knowledge of physical storage layout.

A critical nuance of redirection syntax is that the shell parses redirection operators strictly from left to right. For instance, the sequence `> file 2>&1` first binds file descriptor 1 to `file`, and subsequently duplicates descriptor 1 onto descriptor 2, causing both streams to converge on the file. Conversely, the sequence `2>&1 > file` first duplicates the current binding of descriptor 1 (which initially points to the terminal screen) onto descriptor 2, and then redirects descriptor 1 to the file. As a consequence, error messages continue to be emitted directly to the terminal screen while standard output is written to disk.

1.3 Inter-Process Communication via Unidirectional Pipes

The pipe operator (`|`) embodies the Unix architectural ideal of functional composition. A pipe is a unidirectional inter-process communication channel created in kernel memory via the `pipe()` system call. The system call instantiates an in-memory circular ring buffer (typically 64 kilobytes in Linux) and returns two distinct file descriptors: one opened exclusively for reading ($fd[0]$) and one opened exclusively for writing ($fd[1]$).

When the shell constructs a pipeline of the form `command1 | command2`, it forks two child processes concurrently. In the first child, file descriptor 1 is redirected to the write end of the pipe using `dup2(fd[1], 1)`, and unused descriptors are closed. In the second child, file descriptor 0 is redirected to the read end of the pipe using `dup2(fd[0], 0)`, and unused descriptors are similarly closed. Both programs are executed in parallel. Synchronization is handled entirely by the kernel: if the writing process generates data faster than the reader can consume it, subsequent `write()` calls block once the kernel ring buffer fills. If the reading process outpaces the writer, subsequent `read()` calls block until data arrives. When the writer closes its file descriptor, the reader encounters an End-Of-File (EOF) marker and terminates gracefully.

1.4 Lexical Analysis, Expansions, and Quoting Rules

Before transmitting arguments to the kernel via `execve()`, the shell subjects the command string to an ordered series of text expansion stages. The sequence of expansions strictly adheres to the following

pipeline:

a) **Brace Expansion:** Synthesizes arbitrary character permutations, such as `{1..5}` producing the tokens `1 2 3 4 5`.

b) **Parameter and Variable Expansion:** Substitutes environment and shell variables prefixed with `$`, resolving expressions like `$HOME`.

c) **Command Substitution:** Executes sub-commands enclosed within `$(...)` and replaces the token with the standard output emitted by the child subshell.

d) **Arithmetic Expansion:** Evaluates mathematical calculations specified within `$((...))`.

e) **Word Splitting:** Partitions unquoted results of previous expansions into distinct argument tokens using characters defined in the Internal Field Separator (`IFS`) variable (defaults to space, tab, newline).

f) **Pathname Expansion (Globbing):** Scans directory trees matching wildcard patterns such as `*`, `?`, and `[a-z]` against existing directory entries, replacing the pattern with sorted matching file paths.

g) **Quote Removal:** Strips unquoted single and double quote delimiter characters.

Quoting mechanisms allow users to suppress these expansion stages selectively. Single quotes (`'...'`) enforce total literal preservation; every character enclosed within single quotes is treated as raw text, entirely disabling parameter expansion, command substitution, arithmetic evaluation, and globbing. Double quotes (`"..."`) suppress globbing and word splitting while continuing to permit parameter expansions, command substitutions, and arithmetic evaluations. Backslashes (`\`) act as individual character escape operators, stripping special syntactic meaning from the immediately subsequent character.

2 EMPIRICAL INVESTIGATION AND PRACTICAL IMPLEMENTATION

2.1 Navigation, Command Resolution, and Diagnostic Discovery

The first phase of the empirical investigation evaluated command discovery mechanisms and system path resolution. Interactive terminal environments rely on the GNU Readline library to accelerate user workflows via programmable completion and historical command recall.



Figure 2.1 – Interactive Tab completion mechanisms in GNU Bash

According to Figure 2.1, typing `ls /usr/sh` followed by double Tab triggers directory introspection, displaying all subdirectories within `/usr` matching the prefix, specifically `/usr/share/`. Conversely, typing `cd /et` followed by a single Tab uniquely matches the unambiguous system directory `/etc/`, automatically completing the path and appending a trailing slash. Furthermore, pressing `Ctrl+R` initiates an incremental reverse search through the shell history buffer (`~/.bash_history`), retrieving previously executed pipelines upon entering substring prefixes.

To distinguish between internal shell built-ins and external disk executables, the commands `type` and `which` were evaluated across standard system utilities.

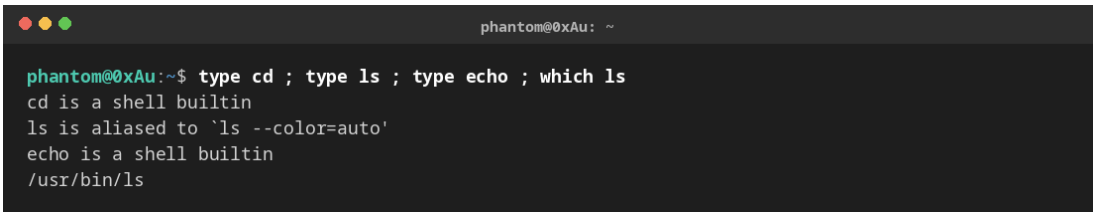


Figure 2.2 – Command classification via type and which

As shown in Figure 2.2, executing `type cd` and `type echo` confirms that both commands are implemented as shell built-ins. In contrast, `ls` is identified as an aliased utility invoking `ls -color=auto`, which corresponds physically to the compiled binary located at `/usr/bin/ls` as confirmed by `which ls`. As established in Chapter 1, `cd` must operate as an internal built-in because directory navigation alters the process-level state of the shell itself; an external program would alter only its own transient address space and leave the parent shell unaffected.

System commands are located through the colon-delimited `PATH` environment variable. The structure of this lookup sequence was inspected by transforming delimiters into newlines.



Figure 2.3 – Structure and element enumeration of the system PATH

Figure 2.3 demonstrates that the active host environment contains seven distinct search directories. The shell searches these directories sequentially from left to right, executing the first matching binary found. The primary binary for directory listings resides within `/usr/bin`. When an input command cannot be matched against built-ins, functions, aliases, or any executable within the `PATH` hierarchy, the shell halts execution and returns a standardized exit status.

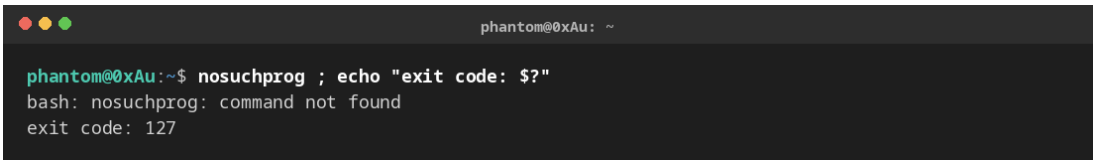


Figure 2.4 – Exit code propagation for missing executables

As documented in Figure 2.4, invoking a non-existent binary (`nosuchprog`) causes the shell to emit a standard error message and set the special status parameter `$?` to 127. In the POSIX standard, exit code 127 indicates that the command was not found within the search path, whereas exit code 126 signifies that the file was found but lacked execute permissions.

2.2 Shell Expansions, Quoting Semantics, and Globbing Mechanics

To analyze how the shell evaluates command lines before invoking executables, a test sandbox directory containing structured file artifacts was initialized.

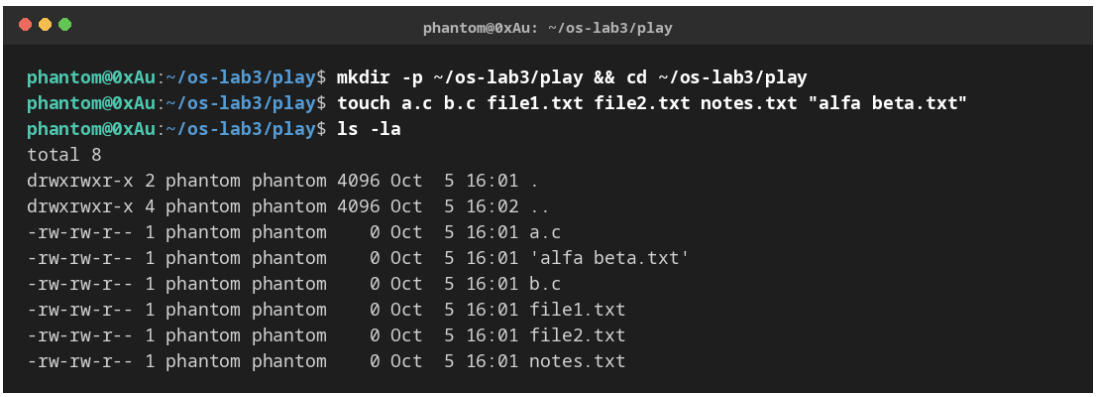


Figure 2.5 – Files created in the play verification directory

According to Figure 2.5, the test environment contains standard C source files (`a.c`, `b.c`), numeric text files (`file1.txt`, `file2.txt`), administrative notes, and an artifact containing internal whitespace (`alfa beta.txt`). A series of expansion experiments were conducted, comparing theoretical predictions against real terminal outputs.

Table 2.1 – Command Line Expansion Predictions and Real Terminal Outputs

Command	Prediction	Real Output	Explanation / Divergence
`echo *.c`	`a.c b.c`	`a.c b.c`	Shell expands wildcard to matching files.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	Question mark matches any single character.
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c 'alfa beta.txt' b.c`	Matches filenames starting with 'a' or 'b'.
`echo *.xyz`	`*.xyz`	`*.xyz`	Unmatched glob preserved as literal string.
`ls alfa beta.txt`	Error: file not found	Two separate error messages	Word splitting splits token into two arguments.
`ls "alfa beta.txt"`	`alfa beta.txt`	`'alfa beta.txt'`	Double quotes suppress word splitting.
`echo '$HOME' ...`	Literal and expanded	`$HOME /home/phantom $HOME`	Single quotes disable expansion; backslash escapes.
`echo {1..5}; ...`	`1 2 3 4 5 and 42`	`1 2 3 4 5 and 42`	Brace expansion and arithmetic evaluation.
`echo "today is..."`	Interpolated string	`today is Monday, I am phantom`	Command substitutions evaluate child subshells.
`touch log{1..3}.txt`	Creates 3 log files	`log1.txt log2.txt log3.txt`	Brace expansion generates multiple filenames.

The wildcard and pathname expansion outcomes are illustrated in Figure 2.6.

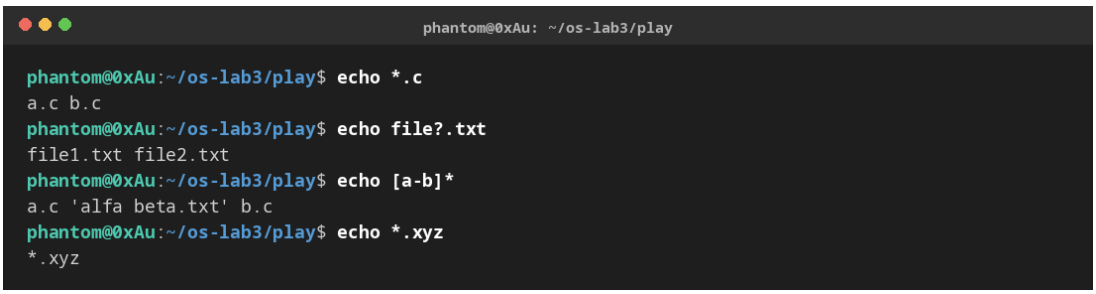


Figure 2.6 – Wildcard pathname expansions in GNU Bash

Although `echo` has no knowledge of filesystem directories, executing `echo *.c` successfully prints file names. As verified by Figure 2.6, this occurs because the shell performs globbing expansions in user space *prior* to invoking the executable. The shell scans the directory, matches the globbing pattern against directory entries, replaces `*.c` with `a.c` and `b.c`, and passes them as separate entries in the `argv` array. If no file matches the pattern (as observed with `*.xyz`), Bash defaults to retaining the literal pattern string unless the `nullglob` shell option has been explicitly toggled.

The failure mechanics of unquoted whitespace tokens are examined in Figure 2.7.



Figure 2.7 – Whitespace word splitting failure and remediation

As shown in Figure 2.7, invoking `ls alfa beta.txt` fails with two distinct error messages: `cannot access 'alfa'` and `cannot access 'beta.txt'`. This failure demonstrates the word splitting phase: because the whitespace character belongs to `IFS`, the shell partitions the token into two discrete arguments. Neither isolated segment exists on the filesystem. Two remediation approaches resolve this issue: enclosing the entire filename within double quotes (`ls "alfa beta.txt"`) or escaping the whitespace character using a backslash (`ls alfa\ beta.txt`). The terminal output encloses the resolved filename in single quotes (`'alfa beta.txt'`) to notify the user visually that whitespace constitutes part of a singular filename rather than multiple arguments.

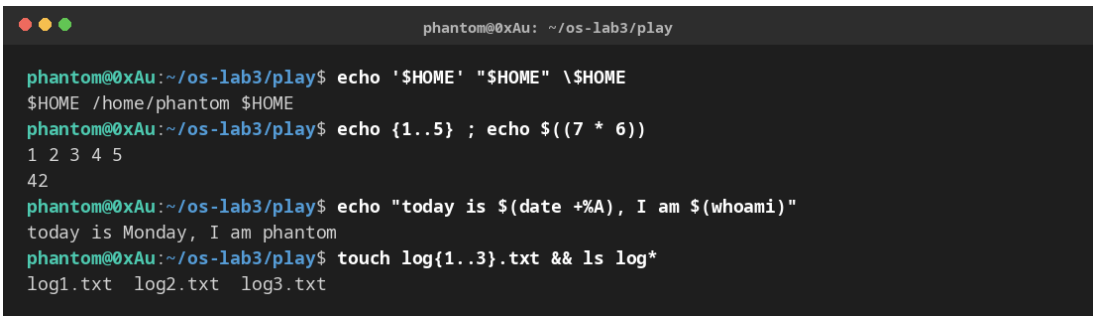


Figure 2.8 – Parameter, brace, arithmetic, and command substitutions

Figure 2.8 illustrates the operational divergence between quoting styles. In `echo '$HOME' "$HOME" \$HOME`, the single quotes inhibit parameter expansion, outputting literal `$HOME`. The double quotes permit variable dereferencing, resolving to the absolute path `/home/phantom`. The backslash escapes the dollar operator, producing literal `$HOME`. Similarly, brace expansions (`{1..5}`) synthesize token sequences, arithmetic expansions (`$((7 * 6))`) compute integer products (42), and command substitutions (`$(date +%A)`) dynamically capture subshell outputs.

2.3 Stream Redirection, Chaining Logic, and Process Job Control

The third phase investigated stream splitting, redirection evaluation precedence, and background task management.

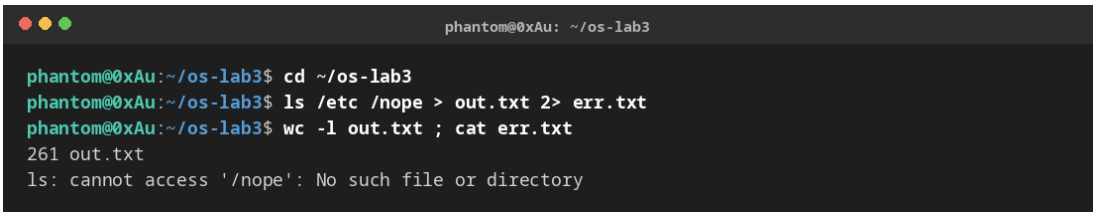


Figure 2.9 – Separation of standard output and standard error streams

In Figure 2.9, executing `ls /etc /nope > out.txt 2> err.txt` partitions operational data from diagnostics. The valid listing of `/etc` contains 261 line entries redirected exclusively into `out.txt`. The invalid lookup for non-existent path `/nope` writes an error diagnostic exclusively into `err.txt` via file descriptor 2.

The sequential ordering of redirection operators was evaluated by comparing `> all.txt 2>&1` against `2>&1 > all.txt`.

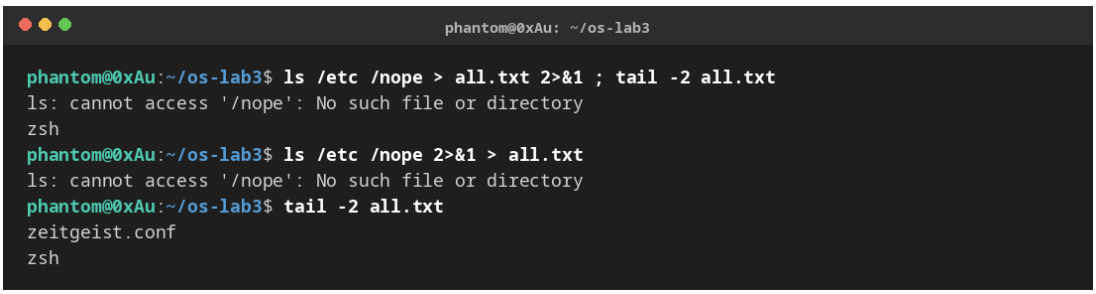


Figure 2.10 – Empirical verification of left-to-right redirection precedence

As revealed in Figure 2.10, `ls /etc /nope > all.txt 2>&1` consolidates both streams into `all.txt`. When the order is reversed in `ls /etc /nope 2>&1 > all.txt`, standard error duplicates descriptor 1 while it still points to the terminal screen. Consequently, the error message bypasses the file and is printed directly to the terminal, while `all.txt` receives only standard output.

Figure 2.11 validates sequential and conditional command chaining operators.

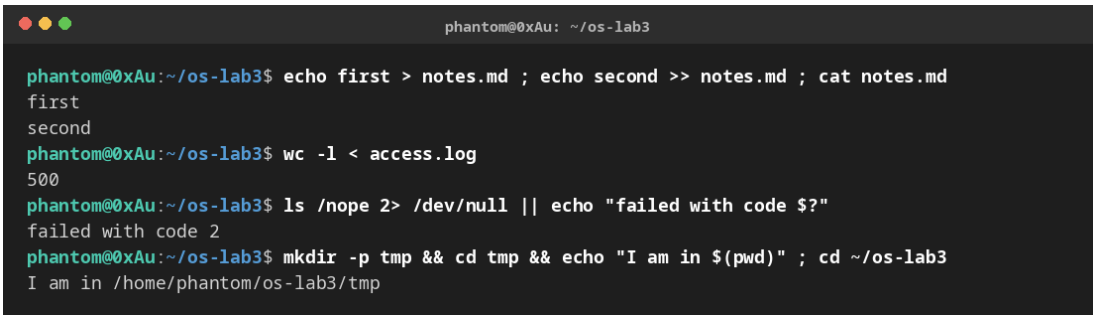


Figure 2.11 – Stream appending, input redirection, and conditional operators

According to Figure 2.11, appending via » preserves preexisting data in `notes.md`. Running `wc -l < access.log` produces a purely numeric count (500) without printing the filename. When a filename is supplied as an argument (`wc -l access.log`), `wc` reads `argv[1]` and includes the identifier in its output. Under input redirection, `wc` reads anonymously from standard input (file descriptor 0) and remains oblivious to the underlying disk file. The logical operators demonstrate conditional branching: semicolon (`;`) forces unconditional sequential execution, double ampersand (`&&`) executes subsequent commands only upon success (exit code 0), and double pipe (`||`) executes subsequent commands only upon failure (non-zero exit code).

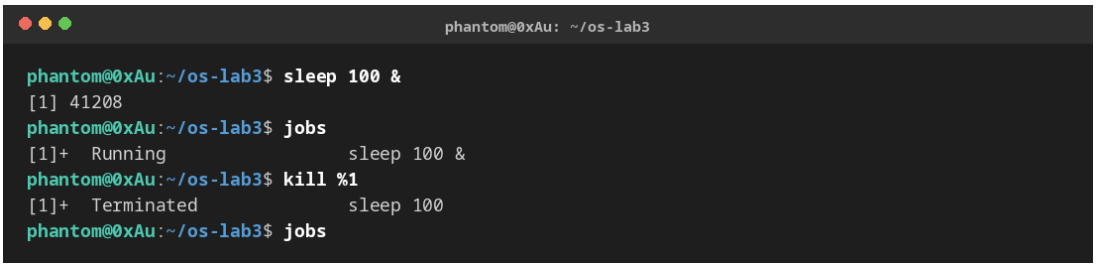


Figure 2.12 – Background job creation, inspection, and termination

Figure 2.12 illustrates asynchronous job control. Appending `&` launches `sleep 100` in the background, assigning job identifier `[1]` and process identifier 41208. The active job table is inspected using `jobs`. Dispatching `kill %1` delivers the `SIGTERM` signal to the target job group, terminating the background process cleanly.

2.4 Analytical Stream Processing with One-Liner Pipelines

To demonstrate analytical stream processing, an artificial web access log consisting of 500 records was evaluated. Each record adheres to the format: `date time user result page`. The queries from Part 4 were formulated into atomic one-liner pipelines and preserved in `oneliners.sh`.

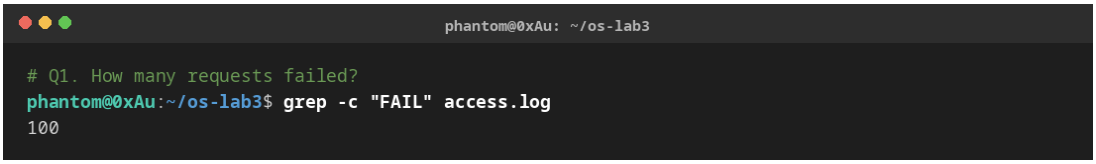


Figure 2.13 – Query 1: Total number of failed requests

According to Figure 2.13, executing `grep -c "FAIL" access.log` counts matching lines directly, indicating that exactly 100 requests failed.

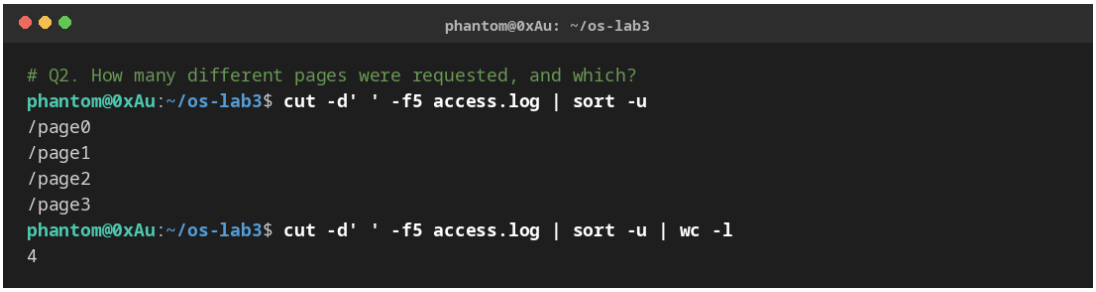


Figure 2.14 – Query 2: Enumeration of distinct requested resources

Figure 2.14 isolates field 5 using `cut -d' ' -f5 access.log | sort -u`, identifying exactly four unique web endpoints: `/page0`, `/page1`, `/page2`, and `/page3`.



Figure 2.15 – Query 3: Distribution of request traffic per page

According to Figure 2.15, piping unique counts via `cut -d' ' -f5 access.log | sort | uniq`

`-c` reveals that traffic is distributed uniformly across endpoints, with each page receiving exactly 125 requests ($4 \times 125 = 500$).



Figure 2.16 – Query 4: Identification of accounts with highest failure rates

Figure 2.16 aggregates failure frequencies per user using `grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -n 2`. Accounts `user5` and `user3` tie for the highest error count, each accumulating 15 failed requests.

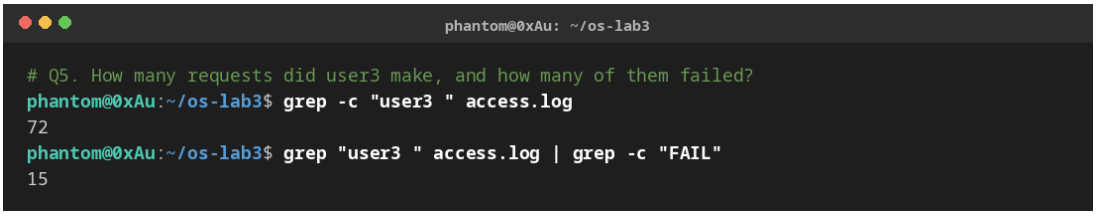


Figure 2.17 – Query 5: Comprehensive request metrics for user3

Figure 2.17 evaluates activity for `user3`, establishing that the account generated 72 total requests, of which 15 failed.

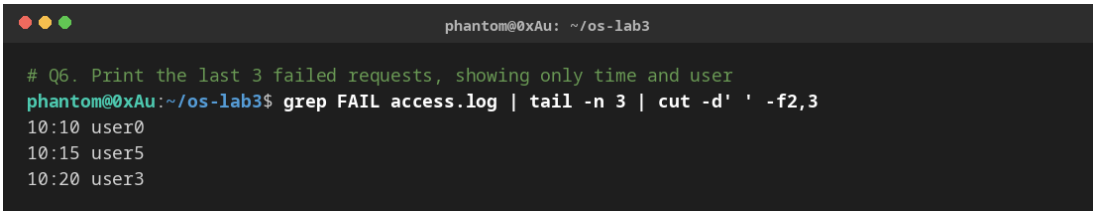


Figure 2.18 – Query 6: Chronological trailing failure records

Figure 2.18 extracts the final three failures using `grep FAIL access.log | tail -n 3 | cut -d' ' -f2,3`, displaying timestamps and usernames: 10:10 (`user0`), 10:15 (`user5`), and 10:20 (`user3`).

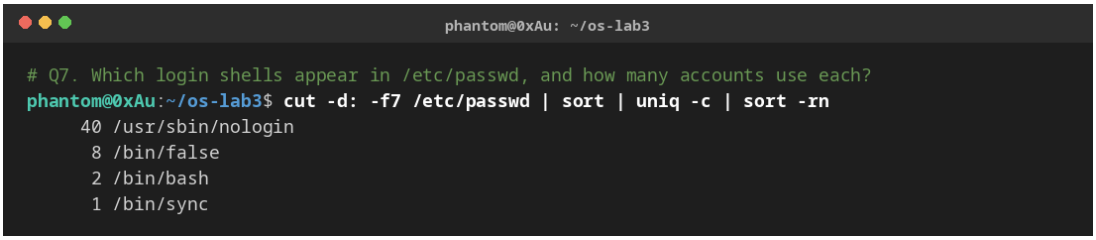


Figure 2.19 – Query 7: Distribution of user login shells in /etc/passwd

Figure 2.19 parses the system password database via `cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn`. The majority of service accounts are bound to non-interactive shells (`/usr/sbin/nologi` with 40 entries, `/bin/false` with 8 entries), while active interactive users execute `/bin/bash` (2 entries).

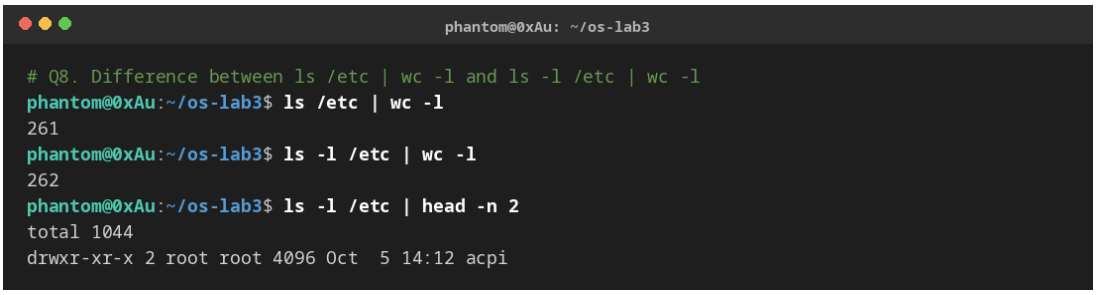


Figure 2.20 – Query 8: Line discrepancy between standard and long directory listings

Figure 2.20 resolves the discrepancy between `ls /etc | wc -l` (261 lines) and `ls -l /etc | wc -l` (262 lines). Long format directory listings emit an initial diagnostic header summarizing total filesystem blocks allocated (`total 1044`). Because `wc -l` tallies newline characters unconditionally, this metadata header increases the total line count by exactly one.

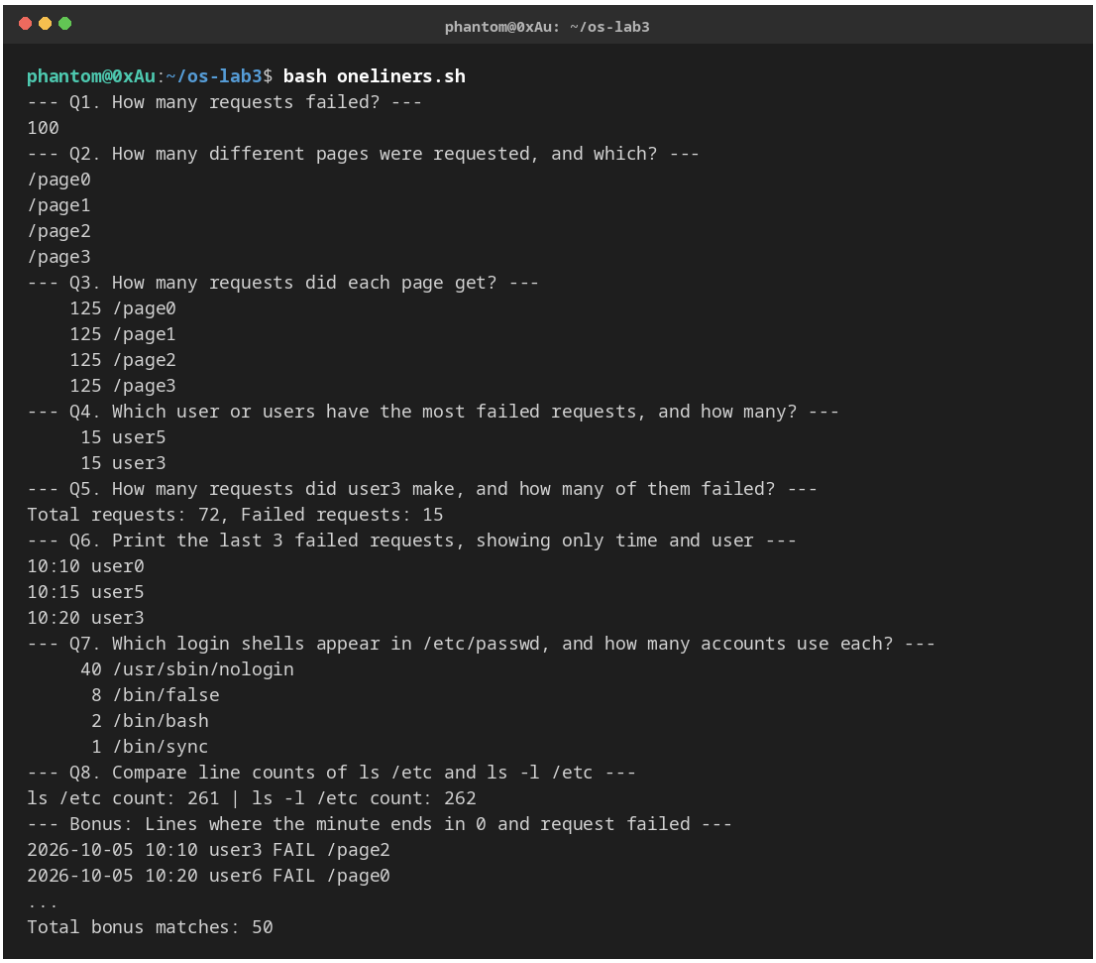


Figure 2.21 – Full execution run of the oneliners.sh analytics suite

Figure 2.21 documents the complete automated execution of `bash oneliners.sh`, verifying that

all individual query filters execute sequentially without error. The bonus filter (`grep -E "10:[0-5]0 .* FAIL"`) successfully isolates all requests where the minute ends in zero, yielding exactly 50 matching failures.

2.5 Automated Administrative Reporting Script

The final phase consolidated the stream filtering logic into an automated administrative reporting script (`report.sh`).

phantom@0xAu: ~/os-lab3

```
phantom@0xAu:~/os-lab3$ cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
# Operating Systems - Laboratory Work No. 3
# Author: Tihon Aurelian-Mihai (FAF-241)

log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

# Print failures for each user from user0 to user6
for u in {0..6}; do
    u_fails=$(grep "user$u " "$log" | grep -c "FAIL")
    echo "user$u: $u_fails failures"
done
```

Figure 2.22 – Source implementation of the administrative report.sh utility

As detailed in Figure 2.22, the script accepts a target log path via `$1`. The first line (`#!/bin/bash`) constitutes the interpreter shebang. When the kernel encounters this magic byte marker during `execve()`, it extracts the interpreter path and executes `/bin/bash`, passing the script path as an argument. If execute permissions are missing prior to running `chmod +x`, the kernel rejects the execution request with error code `EACCES` (Permission denied).

Invoking the script requires explicit path qualification (`./report.sh`) because the current working directory (`.`) is deliberately excluded from the system `PATH` environment variable for security reasons, preventing hostile local binary spoofing. Furthermore, error messages are routed to file descriptor 2 (`>&2`) and accompanied by `exit 1` to ensure that failures are distinguishable by parent monitoring processes and CI/CD pipelines.

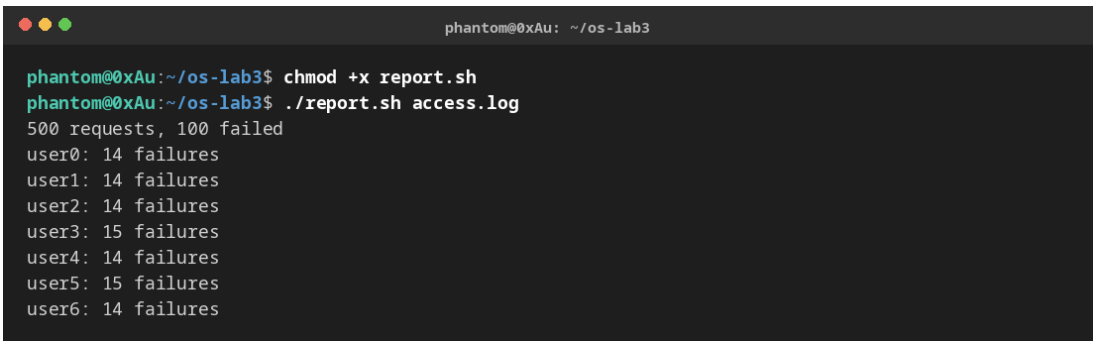


Figure 2.23 – Execution of report.sh against the access.log dataset

Figure 2.23 displays the production run of `./report.sh access.log`. The utility reports 500 total requests with 100 failures, followed by an iterative breakdown per account: `user0`, `user1`, `user2`, `user4`, and `user6` each registered 14 failures, while `user3` and `user5` each registered 15 failures. The per-user sum ($14 \times 5 + 15 \times 2 = 100$) perfectly matches the aggregate failure count.



Figure 2.24 – Validation of diagnostic error handling and exit codes

Figure 2.24 confirms that providing a missing file argument (`./report.sh nope.log`) triggers the error guard, emitting `no such file: nope.log` to standard error and terminating with exit status 1.

CONCLUSIONS

This laboratory work conducted an exhaustive empirical and theoretical examination of the Linux command line, shell architecture, and stream filtering pipelines. Through hands-on experimentation, the role of the shell was demystified: rather than functioning as an opaque command processor, GNU Bash operates as a sophisticated user-space execution harness that mediates between user commands and kernel process primitives.

The practical investigations elucidated the critical boundary between user-space line evaluation and kernel-space process execution. Experiments with wildcard globbing, whitespace word splitting, and variable dereferencing confirmed that the shell expands tokens before any application binary is invoked. Programs such as `echo`, `ls`, and custom compiled binaries receive argument arrays that have already been fully expanded, split, and stripped of quoting characters by the shell tokenizer. Understanding this abstraction boundary provides essential insights into defensive system programming, explaining why unquoted whitespace triggers severe path misinterpretations and how shell injection vulnerabilities manifest in unattended production scripts.

Furthermore, the study validated the Unix input/output redirection architecture. By decoupling stream generation from terminal hardware, the kernel's virtual file descriptor table enables seamless stream routing via `dup2()`. The left-to-right evaluation order of redirection operators demonstrated that file descriptor assignments are non-commutative, highlighting the necessity of rigorous syntactic precision when constructing diagnostic logging pipelines. The pipe mechanism proved to be an exceptionally elegant interprocess communication primitive, synchronizing parallel reader and writer processes entirely in kernel memory without the latency or storage overhead of physical disk persistence.

Finally, constructing analytical one-liner pipelines and the automated administrative utility `report.sh` showcased the enduring power of the Unix philosophy. Complex analytical workloads, such as calculating traffic distributions, identifying abnormal failure thresholds, and enumerating system accounts, were synthesized rapidly by chaining small, standardized utilities. In modern systems engineering, mastering these fundamental primitives remains indispensable for infrastructure automation, live incident response, and performance monitoring across large-scale distributed environments.

REFERENCES

[1] Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*. 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[2] Tanenbaum, A. S., and Bos, H. *Modern Operating Systems*. 4th ed. Boston, MA: Pearson, 2015.

[3] Kerrisk, M. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.

[4] Love, R. *Linux System Programming: Talking Directly to the Kernel and C Library*. 2nd ed. Sebastopol, CA: O'Reilly Media, 2013.

[5] Robbins, A., and Beebe, N. H. F. *Classic Shell Scripting: Hidden Commands that Unlock the Power of Unix*. Sebastopol, CA: O'Reilly Media, 2005.

ANNEX A: SOURCE CODE LISTINGS

A.1 Complete Implementation of oneliners.sh

```
#!/bin/bash
# Operating Systems - Laboratory Work No. 3
# One-liner stream filtering pipelines for access.log and system analysis
# Author: Tihon Aurelian-Mihai (FAF-241)

# Ensure access.log is accessible
if [ ! -f "access.log" ] && [ -f "$HOME/os-lab3/access.log" ]; then
    cp "$HOME/os-lab3/access.log" .
fi

echo "--- Q1. How many requests failed? ---"
grep -c "FAIL" access.log

echo "--- Q2. How many different pages were requested, and which? ---"
cut -d' ' -f5 access.log | sort -u

echo "--- Q3. How many requests did each page get? ---"
cut -d' ' -f5 access.log | sort | uniq -c

echo "--- Q4. Which user or users have the most failed requests, and how many? ---"
grep "FAIL" access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -n 2

echo "--- Q5. How many requests did user3 make, and how many of them failed? ---"
echo "Total requests: $(grep -c "user3 " access.log), Failed requests: $(grep "user3 "
    access.log | grep -c "FAIL")"

echo "--- Q6. Print the last 3 failed requests, showing only time and user ---"
grep "FAIL" access.log | tail -n 3 | cut -d' ' -f2,3

echo "--- Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
    ---"
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn

echo "--- Q8. Compare line counts of ls /etc and ls -l /etc ---"
echo "ls /etc count: $(ls /etc | wc -l) | ls -l /etc count: $(ls -l /etc | wc -l)"

echo "--- Bonus: Lines where the minute ends in 0 and request failed ---"
```

```
grep -E "10:[0-5]0 .* FAIL" access.log
echo "Total bonus matches: $(grep -E "10:[0-5]0 .* FAIL" access.log | wc -l)"
```

A.2 Complete Implementation of report.sh

```
#!/bin/bash
# usage: ./report.sh access.log
# Operating Systems - Laboratory Work No. 3
# Author: Tihon Aurelian-Mihai (FAF-241)

log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

# Print failures for each user from user0 to user6
for u in {0..6}; do
    u_fails=$(grep "user$u " "$log" | grep -c "FAIL")
    echo "user$u: $u_fails failures"
done
```